

FYZIKÁLNÍ PRAKTIKUM

Fyzikální praktikum 1

Zpracoval: Miroslav Gašpar

Naměřeno: 24. dubna 2025

Obor: Fyzika-nanotechnologie

Ročník: I

Semestr: II

Testováno:

Úloha č. 10: Tepelná vodivost' pevných látek

$$T = 23,6^{\circ}\text{C}$$

$$p = 988 \text{ hPa}$$

$$\varphi = 46,7 \text{ rel. \%}$$

1. Zadanie

- Numericky riešte rovnicu vedenia tepla pre zvolené počiatkové a okrajové podmienky v medenej tyči.
- Experimentálne realizujte zvolené počiatkové a okrajové podmienky.
- S vhodným časovým a priestorovým rozlíšením merajte vývoj rozloženia teploty pozdĺž tyče.
- Určte teplotnú vodivosť kovovej tyče s využitím numerickej simulácie tak, aby sa dosiahla maximálna zhoda s experimentálnymi dátami.

2. Teória

Tepelná vodivosť je materiálový parameter, ktorý charakterizuje schopnosť látky viesť teplo. V tejto úlohe sa zameriame na vedenie tepla v jednorozmernom systéme (tyči) a na určenie teplotnej vodivosti numerickou simuláciou.

2.1. Rovnica vedenia tepla

Ak sa rozloženie teploty v tyči mení s časom, prenos tepla popisuje rovnica vedenia tepla. Pre jednorozmerný prípad má tvar (1). V tomto vzťahu $t(x, \tau)$ je teplota v mieste x a čase τ , ρ je hustota materiálu a c je jeho merná tepelná kapacita. Výraz $a = \lambda/(\rho c)$ sa nazýva súčiniteľ teplotnej vodivosti (nezamieňať so súčiniteľom tepelnej vodivosti λ). Tento súčiniteľ určuje, ako rýchlo sa mení rozloženie teploty v materiáli.

$$\frac{\partial t(x, \tau)}{\partial \tau} = \frac{\lambda}{\rho c} \frac{\partial^2 t(x, \tau)}{\partial x^2} \quad (1)$$

Nakoľko v jednorozmernej rovnici vedenia tepla vystupujú tri derivácie. Potrebujeme jednu počiatkovú podmienku a dve okrajové. Ako počiatkovú podmienku si stanovíme hodnoty z termočlánkov v čase $t = 0$ a ako okrajové podmienky použijeme hodnoty namerané termočlánkami na okrajoch tyče.

V prípade ak za dostatočne dlhú dobu pri konštantnom výkone chladiča a ohrievača dôjde k ustáleniu rozloženia teploty v tyči v čase, dostávame rovnicu (2).

$$0 = \frac{\partial^2 t(x, \tau)}{\partial x^2} \quad (2)$$

Z nej jednoduchou úvahou prídeme k záveru, že v ustálenom stave je teplota lineárnou funkciou premennej x (pozície na tyči).

2.2. Numerické riešenie rovnice vedenia tepla

Pre naše numerické riešenie, použijeme metódu konečných diferencií. Pri tejto metóde sa spojité priestorové a časové premenné diskretizujú. Tyč rozdelíme na N intervalov dĺžky Δx a časový vývoj sledujeme v krokoch $\Delta\tau$. Teplotu v bode $x_k = k\Delta x$ a čase $\tau_j = j\Delta\tau$ označíme t_k^j . Parciálne derivácie v rovnici (1) nahradíme diferenciami. Časovú deriváciu môžeme aproximovať ako (3).

$$\frac{\partial t}{\partial \tau} \approx \frac{t_k^{j+1} - t_k^j}{\Delta\tau} \quad (3)$$

Druhú priestorovú deriváciu aproximujeme ako (4).

$$\frac{\partial^2 t}{\partial x^2} \approx \frac{t_{k-1}^j - 2t_k^j + t_{k+1}^j}{(\Delta x)^2} \quad (4)$$

Dosadením týchto aproximácií do rovnice (1) a úpravou získame explicitnú diferenčnú schému pre výpočet teploty v aktuálnom časovom kroku j z hodnôt v predchádzajúcom kroku $j-1$ vzťahom (5).

$$t_k^j = t_k^{j-1} + \Delta\tau \frac{\lambda}{\rho c} \frac{t_{k-1}^{j-1} - 2t_k^{j-1} + t_{k+1}^{j-1}}{\Delta x^2} \quad (5)$$

Pre stabilitu numerického riešenia explicitnou metódou musí byť splnená podmienka (6).

$$\Delta\tau \leq \frac{\rho c}{\lambda} \frac{\Delta x^2}{2} \quad (6)$$

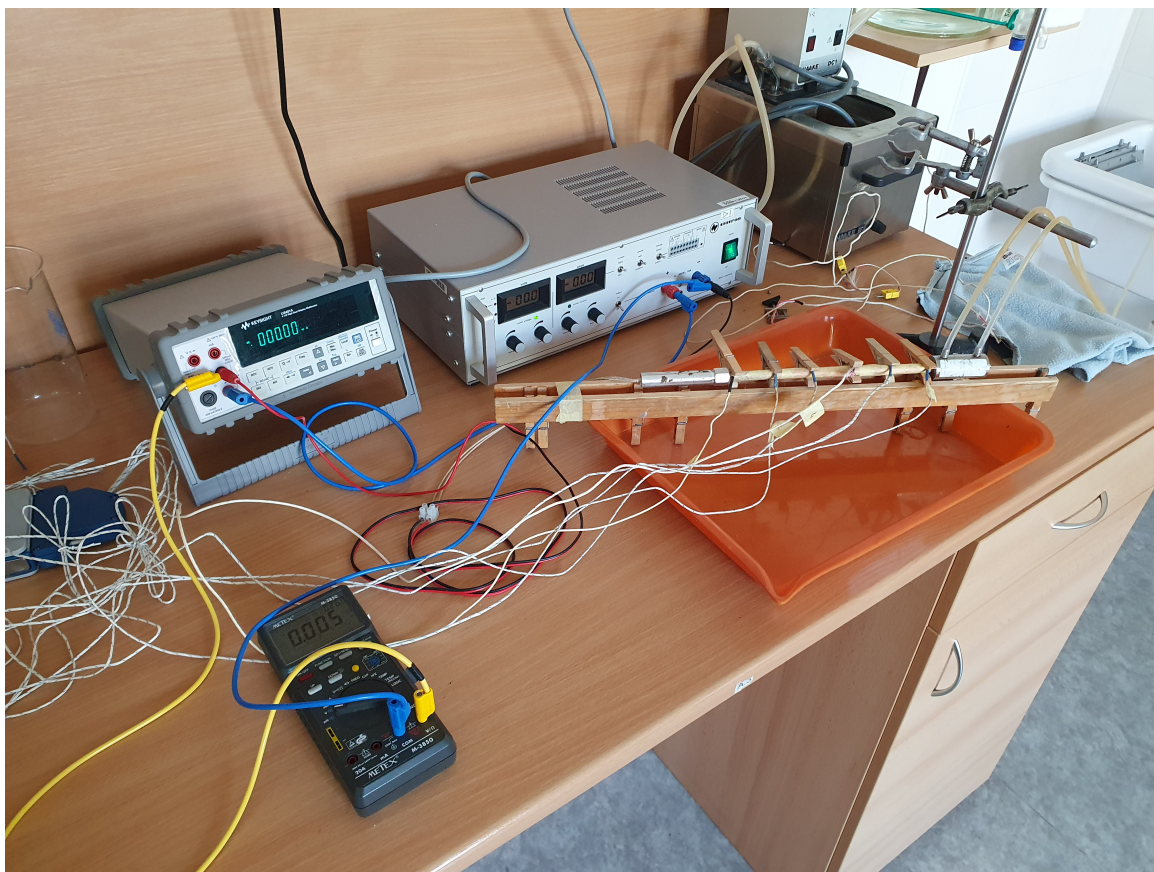
3. Pomôcky

Medená tyč s priemerom 99,8(0,06) mm a dĺžkou 301(0,3) mm. Sada termočlánkov typu K na meranie rozloženia teploty so štandardnou neistotou 2,2 °C alebo 0,75 %, platí väčšia hodnota. Meracia karta NI 9212 s počítačom a predpripraveným programom na záznam dát v softvéri LabView. Elektrické výhrevné teliesko Omega CIR-1014/120 ako ohrievač. Kvapalinový chladiaci element s pripojením na vodovod ako chladič. Zdroj Straton pre napájanie ohrievača. Ampérmeter Keysight U3401A a voltmeter Metex M-3850 pre určenie výkonu ohrievača. Pravítka pre určenie dĺžky tyče a rozmiestnenia termočlánkov s najmenším dielikom 1 mm. Posuvné meradlo s najmenším dielikom 0,02 mm pre určenie priemeru tyče. Softvér VScode s rozšíreniami Python a Pylance pre programovanie v jazyku Python a prácu s Jupyter notebooks.

4. Meranie

Pre zrealizovanie nášho experimentu sme si pripravili aparatúru (Obr. 1), na medenú tyč sme si z jednej strany umiestnili elektrické výhrevné teliesko a z druhej strany kvapalinový chladič pripojený na vodovod, medzi nimi nám zostal voľný úsek dlhý približne 21 cm. Následne sme zahriatím rukou overili, ktoré z termočlánkov pripojených k meracej karte sú funkčné, boli to 0, 2, 3, 4, 5, 7 (v dátových súboroch je v prvom stĺpci čas, preto indexovanie v Pythone o 1 vyššie). Voľný úsek sme pre zamedzenie parazitických prúdov omotali papierovou páskou, následne sme v rozstupoch 4,2 cm štipcami pripevnili 6 termočlánkov, rozstupy sme merali pomocou pravítka, ako počiatok osi x sme uvažovali stranu pri ohrievači. Ohrievač sme pripojili ku zdroju a metódou A sme k nemu zapojili ampérmeter a voltmeter pre odmeranie výkonu ohrievača.

Najprv sme spustili chladič bez ohrievača, tyč sme chladili, dokým sa v nej neprestala meniť teplota v závislosti na čase, nastal ustálený stav. Následne sme spustili ohrievač a začali naše meranie (príloha *data.txt*), tyč sme ohrievali konštantným výkonom $P = 30,9(1)V \cdot 0,329(1)A = 10,166(3) W$, zároveň sme pokračovali v chladení konštantným prietokom vody z druhej strany (príloha *ohrev.txt*, prvých 1364 riadkov súboru *data.txt*). Po vzniku nového ustáleného stavu sme vypli ohrievač a schladili tyč na hodnoty predošlého ustáleného stavu (príloha *chladnutie.txt*, zvyšné riadky súboru *data.txt*).



Obr. 1: Zostavená aparatúra pred začiatkom merania

5. Simulácie

Pre spracovanie nami nameraných dát sme naprogramovali dve simulácie, prvá nekladie žiadne nároky na vstupné dáta. Pri druhej využívame náš poznatok z rovnice (2), že pri ustálenom stave je teplota v tyči lineárnou funkciou pozície na tyči, to sme použili na zinterpolovanie počiatočnej podmienky a následné zjemnenie delenia pozície (zmenšenie dx). Avšak stále musíme brať na zreteľ podmienku stability (6) a neprekročiť ju prílišným nepomerom medzi delením času a pozície.

5.1. Simulácia bez interpolácie

Pre túto simuláciu sme si v kóde nižšie zadefinovali niekoľko funkcií, ktorých volaním do vstupných dát naftujeme simuláciu s optimálnym súčiniteľom tepelnej vodivosti, z ktorého následne dopočítame teplotnú vodivosť, hustotu medi uvažujeme $\rho = 8960 \text{ kgm}^{-3}$ [1], mernú tepelnú kapacitu $c = 383 \text{ kgm}^{-3}$ [2]. Naša simulácia vo výpočte zahŕňa aj štandardnú neistotu vstupných dát, výsledok vracia so štandardnou neistotou.

```

1  from mpl_toolkits.mplot3d import Axes3D
2  import matplotlib.ticker as ticker
3  import matplotlib.pyplot as plt
4  from scipy.optimize import curve_fit
5  import numpy as np
6  import pandas as pd
7  %matplotlib inline
8
9  import locale # zabezpeči použitie desatinnej ciarky miesto bodky v grafoch
10 locale.setlocale(locale.LC_NUMERIC, "de_DE")

```

```

11 plt.rcParams['axes.formatter.use_locale'] = True
12
13 data = t = np.array([[], []])
14 poc_hodnoty = tau = np.array([])
15 poz_clanky = np.array([0, 0.042, 0.084, 0.126, 0.168, 0.21]) # rozmiestnenie clankov na tycki
16
17 c = 383 # merna tepelna kapacita medi
18 rho = 8960 # hustota medi
19
20 delta_x1 = 0.042 # vzdialenost termoclankov na tycki
21 def simulacia1(vystup, koef):
22     sim = np.zeros_like(t)
23     sim[:,0] = poc_hodnoty # pociatocna podmienka
24     for j in range(1,len(tau)):
25         sim[0,j] = t[0,j] # okrajove podmienky
26         sim[-1,j] = t[5,j]
27         for k in range(1,len(sim)-1):
28             delta_tau = tau[j] - tau[j-1] # casovy krok
29             # vzťah zo zadania
30             citatel = sim[k-1,j-1]-2*sim[k,j-1]+sim[k+1,j-1]
31             sim[k,j] = sim[k,j-1] + delta_tau*koef*citatel/delta_x1**2
32     result = sim
33     if vystup[0] == 0: # rozlisenie dvoch vystupov, pre fitovanie a pre vykreslenie grafu
34         result = sim.flatten()
35     return result
36
37 def podmienka(koef, delta_x, delta_tau): # overenie podmienky stability zo zadania
38     if delta_tau>=1/koef*delta_x**2/2:
39         print('Podmienka nie je splnená!!!')
40     else: print('Podmienka je splnená.')
41
42 def sigma(y_data): # vypocet absolutnej neistoty pre kazdu hodnotu nameranu termoclankami
43     # neistota, bud 0,75% alebo 2,2°C, ta ktora je vacsia
44     sigma_rel = 0.0075
45     sigma_abs1 = np.ones_like(y_data)*0.041*2.2
46     sigma_abs2 = sigma_rel * np.abs(y_data)
47
48     return np.maximum(sigma_abs1, sigma_abs2) # pre kazdu hodnotu vrati vyssiu neistotu

```

Pre volanie funkcií a inicializáciu simulácie sme používali nasledovný kód, vždy s dátami, do ktorých sme chceli fitovať.

```

1 # simulacia pre ohrievanie aj ochladzovanie
2 # nacitanie dat
3 data = pd.read_csv('data.txt', sep = '\t', decimal=',', header=None)
4 poc_hodnoty = np.array([data[1][0],data[3][0],data[4][0],
5                         data[5][0],data[6][0],data[8][0]], dtype=float)
6 tau = np.array(data[0], dtype=float)
7 t = np.array([data[1],data[3],data[4],data[5],data[6],data[8]], dtype=float)
8 delta_tau = (tau[-1]-tau[0])/len(tau)
9
10 y_data = t.flatten() # do curve_fit musia vstupovat 1D matice
11 popt, pcov = curve_fit(simulacia1, np.zeros_like(y_data), y_data,
12                        p0=[1.16e-4], # odhad teplotnej vodivosti - tabulkova hodnota

```

```

13         sigma=sigma(y_data), # standardna neistota merania
14         absolute_sigma=True)
15
16 perr = np.sqrt(np.diag(pcov)) # standardna neistota
17 podmienka(popt[0],delta_x1,delta_tau)
18 print(popt[0], ' ',perr[0]) # hodnota teplotnej vodivosti
19 print(popt[0]*c*rho, ' ',perr[0]*c*rho) # hodnota tepelnej vodivosti

```

Prvé sme fitovali dáta celého merania, ohrev aj chladnutie. Výsledná tepelná vodivosť bola $\lambda_{celkom} = 290(9) \text{ Wm}^{-1}\text{K}^{-1}$. Pre dáta v časti ohrievania bola $\lambda_{ohr.} = 193(6) \text{ Wm}^{-1}\text{K}^{-1}$. Pre chladnutie bola $\lambda_{chl.} = 410(30) \text{ Wm}^{-1}\text{K}^{-1}$. Pri všetkých simuláciách bola podmienka splnená.

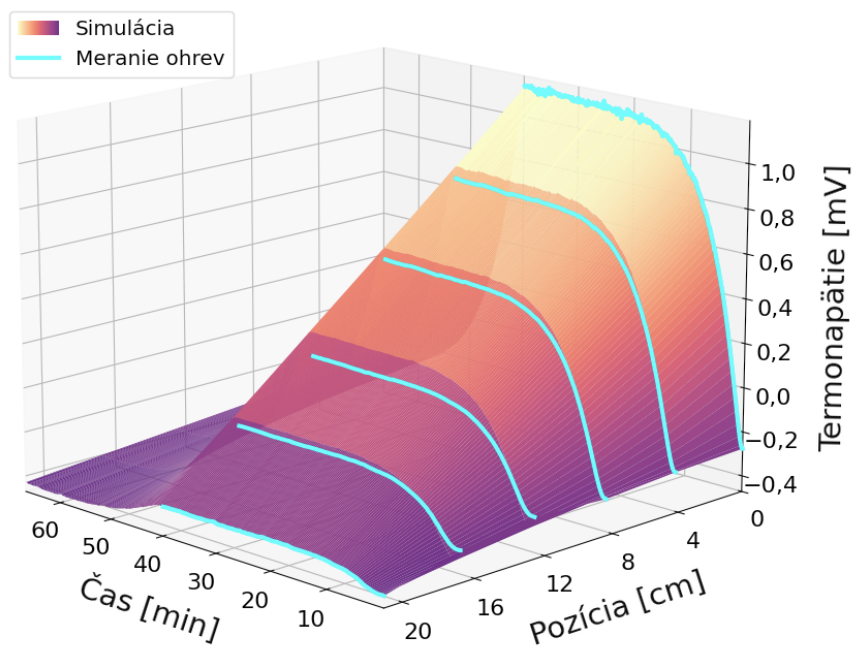
Dáta z celého merania (ohrev a chladnutie spolu) sme s ich príslušnou simuláciou vizualizovali do trojrozmerného grafu kódom nižšie.

```

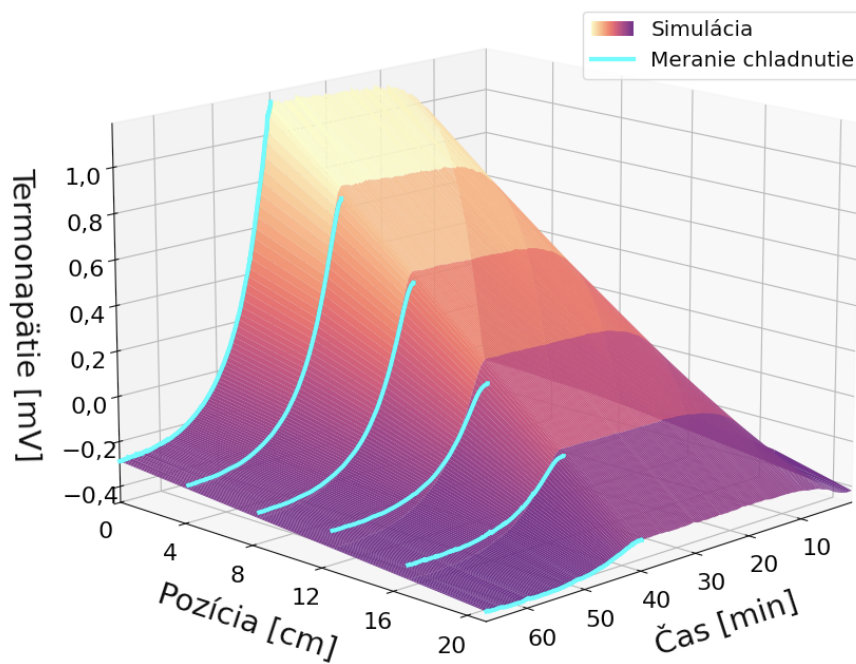
1  # vytvorenie 3D grafu
2  T_grid, X_grid = np.meshgrid(tau/60, poz_clanky*100) # definicny obor
3  zdata = simulacia1(np.ones_like(y_data),popt[0]) # obor hodnot
4
5  fig_3d = plt.figure(figsize=(10, 10))
6  ax_3d = fig_3d.add_subplot(111, projection='3d',computed_zorder=False)
7
8  # vykreslenie povrchu
9  surf = ax_3d.plot_surface(T_grid, X_grid, zdata, cmap='magma',
10                             vmin=-1, edgecolor='none', rstride=1, label='Simulácia')
11 # vykreslenie nameranych dat
12 for i in range(6):
13     x = tau[1363:2288] / 60
14     y = poz_clanky[i]*100
15     z = t[i, 1363:2288]
16
17     ax_3d.plot(x, y, z, color='cyan', linewidth=3)
18
19 # priehladna ciara pre potreby legendy
20 ax_3d.plot(0, 0, 0, color = 'cyan', linewidth = '3', alpha=1, label='Meranie chladnutie')
21
22 # nastavenie popisov osí a designu
23 ax_3d.set_xlabel('Čas [min]', fontsize=21, labelpad=15)
24 ax_3d.set_ylabel('Pozícia [cm]', fontsize=21, labelpad=15)
25 ax_3d.set_zlabel('Termonapätie [mV]', fontsize=21, labelpad=15)
26 ax_3d.tick_params(labelsize=16)
27 ax_3d.yaxis.set_major_locator(ticker.MultipleLocator(base=4))
28
29 ax_3d.set_xlim(tau.max()/60, tau.min()/60)
30 ax_3d.set_ylim(poz_clanky.max()*100, poz_clanky.min()*100)
31 ax_3d.view_init(elev=15, azim=-135) # poloha pozorovateľa
32 plt.legend(fontsize='x-large',loc='upper right', bbox_to_anchor=(0.99, 0.9)) # legenda
33 plt.savefig('graf1.png',bbox_inches='tight',pad_inches=1) # ulozenie grafu
34 plt.show()

```

Pre zvýšenie prehľadnosti sme vizualizovali z dvoch uhlov, výsledkami sú tieto grafy (Obr. 2, 3).



Obr. 2: Dáta z čidiel pri ohreve (tyrkysové krivky) a celková simulácia (farebná plocha)



Obr. 3: Dáta z čidiel pri chladnutí (tyrkysové krivky) a celková simulácia (farebná plocha)

Na vizualizácii ohrevu (Obr. 2) je ľahko viditeľná odchýlka fitu od nameraných dát v oblasti s najvyššími teplotami, simulácia tu teplotne prevyšuje namerané dáta. Na vizualizácii chladnutia (Obr. 3) je odchýlka badateľná v oblasti s najnižšími teplotami, a to v časti bližšej k chladiču, kedy namerané dáta teplotami prevyšujú hodnoty simulácie.

5.2. Simulácia s interpoláciou počiatočnej podmienky

Pri tejto simulácii je nutnou požiadavkou na vstupné dáta, aby začínali v tepelnom ustálenom stave. Takéto stavy sme počas nášho merania mali tri, na začiatku, po zohriatí a po schladení.

Táto simulácia je modifikáciou predošlej. Rovnako ako pri nej sme si napred zadefinovali niekoľko funkcií, ktorých volaním sme do vstupných dát fitovali simuláciu. Naša simulácia opäť započítava aj vracia štandardnú neistotu. Zadefinované funkcie sú v tomto kóde.

```
1 import matplotlib.pyplot as plt
2 from scipy.optimize import curve_fit
3 from scipy.stats import linregress
4 import numpy as np
5 import pandas as pd
6 %matplotlib inline
7
8 import locale # zabezpeci pouzitie desatinnej ciarky miesto bodky v grafoch
9 locale.setlocale(locale.LC_NUMERIC, "de_DE")
10 plt.rcParams['axes.formatter.use_locale'] = True
11
12 data = t = np.array([[], []])
13 poc_hodnoty = tau = np.array([])
14 poz_clanky = np.array([0, 0.042, 0.084, 0.126, 0.168, 0.21])
15
16 c = 383 # merna tepelna kapacita medi
17 rho = 8960 # hustota medi
18
19 # ziskanie lin. predpisu pre interpoláciu a vykreslenie grafu
20 def fit_plot(poz, hodnoty):
21     slope, intercept, r, p, stderr = linregress(poz, hodnoty) # metoda najmensich stvorcov
22     # vytvorenie grafu
23     fig, ax = plt.subplots(figsize=(9,8))
24     ax.set_xlabel('Poloha na tyči [cm]')
25     ax.xaxis.label.set_fontsize(21)
26     ax.set_ylabel('Termonapätie [mV]')
27     ax.yaxis.label.set_fontsize(21)
28     ax.tick_params(labelsize=16)
29     x_ticks = np.arange(0, 22, step=4.2)
30     ax.set_xticks(x_ticks)
31     step = round(((max(hodnoty)-min(hodnoty))/5),2)
32     y_ticks = np.arange(round(min(hodnoty),2), max(hodnoty)+1, step=step)
33     ax.set_yticks(y_ticks)
34
35     ax.scatter(poz*100, hodnoty, label='Dáta z termočlánkov')
36     popis = (r'Lineárny fit:  $\tan\alpha = {} + f \cdot \{slope:.2f\}(\{stderr:.2f\})'$ '.replace('.', ','))
37     ax.plot(poz*100, slope*poz+intercept,
38             label=popis,
39             color='red')
40     plt.legend(fontsize='x-large', loc='lower left')
41     plt.show()
```

```

42
43     return(slope, intercept, stderr)
44
45 n_dx = 10 # pocet diskretnych hodnot pri diferencovani
46 delenie = np.linspace(poz_clanky[0],poz_clanky[-1],n_dx)
47 delta_x2 = delenie[1] - delenie[0] # priestorovy krok
48 def simulacia2(vystup, koef):
49     slope, intercept, r, p, stderr = linregress(poz_clanky,poc_hodnoty)
50     sim = np.zeros((len(delenie), len(tau)))
51     sim[:,0] = np.array([i*slope+intercept for i in delenie]) # pociatocna podmienka
52     for j in range(1,len(tau)):
53         sim[0,j] = t[0,j] # okrajove podmienky
54         sim[-1,j] = t[5,j]
55         for k in range(1,len(sim)-1):
56             delta_tau = tau[j] - tau[j-1] # casovy krok
57             # vzťah zo zadania
58             citatel = sim[k-1,j-1]-2*sim[k,j-1]+sim[k+1,j-1]
59             sim[k,j] = sim[k,j-1] + delta_tau*koef*citatel/delta_x1**2
60     if vystup[0] == 0:
61         result = np.zeros_like(t)
62         # pre fit potrebujeme rovnake priestorove delenie ako pri merani dat
63         # vyberie data s hodnotami najblizsiami k poziciam termoclankov
64         for i, n in zip(poz_clanky, range(0,len(result))):
65             result[n,:] = (sim[np.argmin(np.abs(delenie - i)),:])
66     return result.flatten()
67
68 def podmienka(koef, delta_x, delta_tau): # overenie podmienky stability zo zadania
69     if delta_tau>=1/koef*delta_x**2/2:
70         print('Podmienka nie je splnená!!! Zmeň n_dx!!!')
71     else: print('Podmienka je splnená.')
72
73 def sigma(y_data): # vypocet absolutnej neistoty pre kazdu hodnotu nameranu termoclankami
74     # neistota, bud 0,75% alebo 2,2°C, ta ktora je vacsia
75     sigma_rel = 0.0075
76     sigma_abs1 = np.ones_like(y_data)*0.041*2.2
77     sigma_abs2 = sigma_rel * np.abs(y_data)
78
79     return np.maximum(sigma_abs1, sigma_abs2) # pre kazdu hodnotu vrati vyssiu neistotu

```

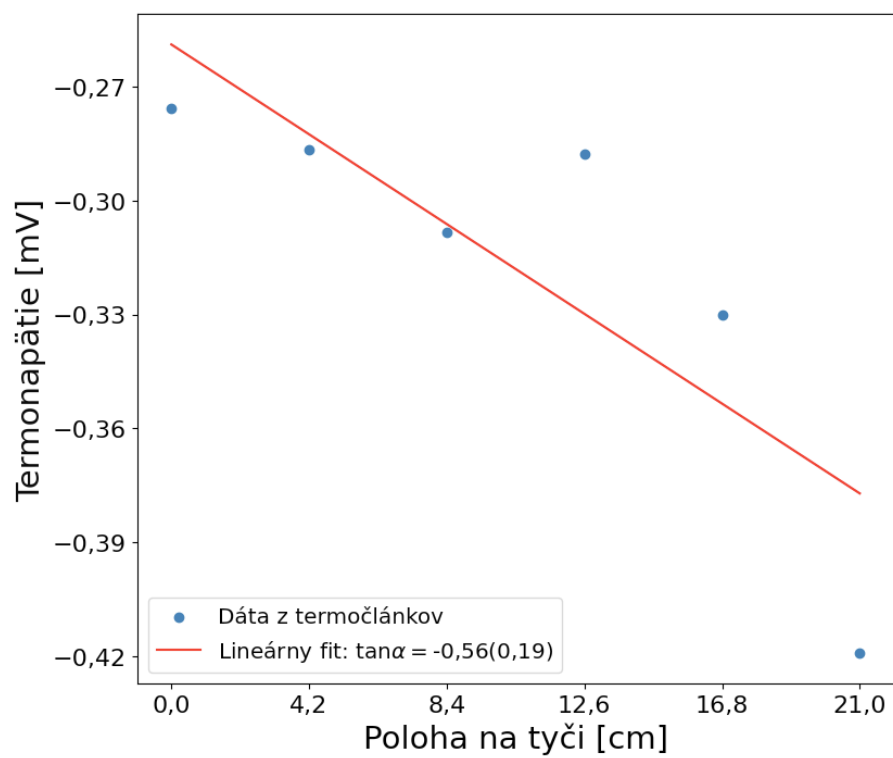
Pred samotným fitovaním sme si vykreslili hodnoty v každom ustálenom stave aj s lineárnym fitom, aby sme preverili, či teplota ozaj spĺňa lineárnu závislosť, alebo sú efekty spojené s výmenou tepla s okolím priveľké. Na to sme zavolali jednu zo zadaných funkcií nasledovne.

```

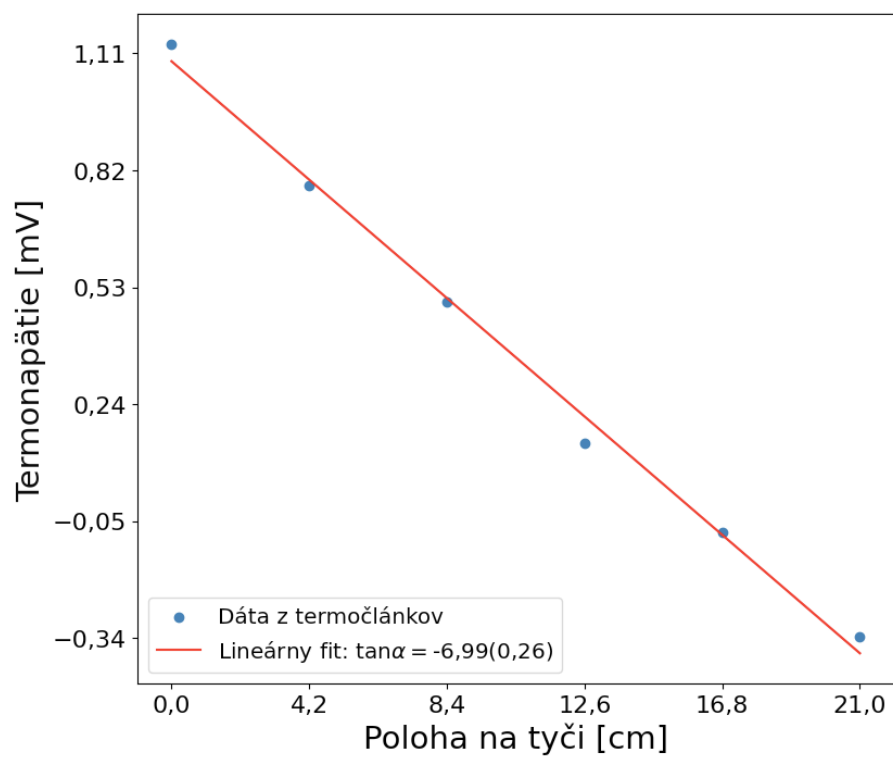
1 # vykreslenie rovnovazneho stavu
2 data = pd.read_csv('data.txt', sep = '\t', decimal=',',header=None)
3 poc_hodnoty = np.array([data[1][0],data[3][0],data[4][0],
4                         data[5][0],data[6][0],data[8][0]], dtype=float)
5
6 slope, intercept, stderr = fit_plot(poz_clanky, poc_hodnoty)

```

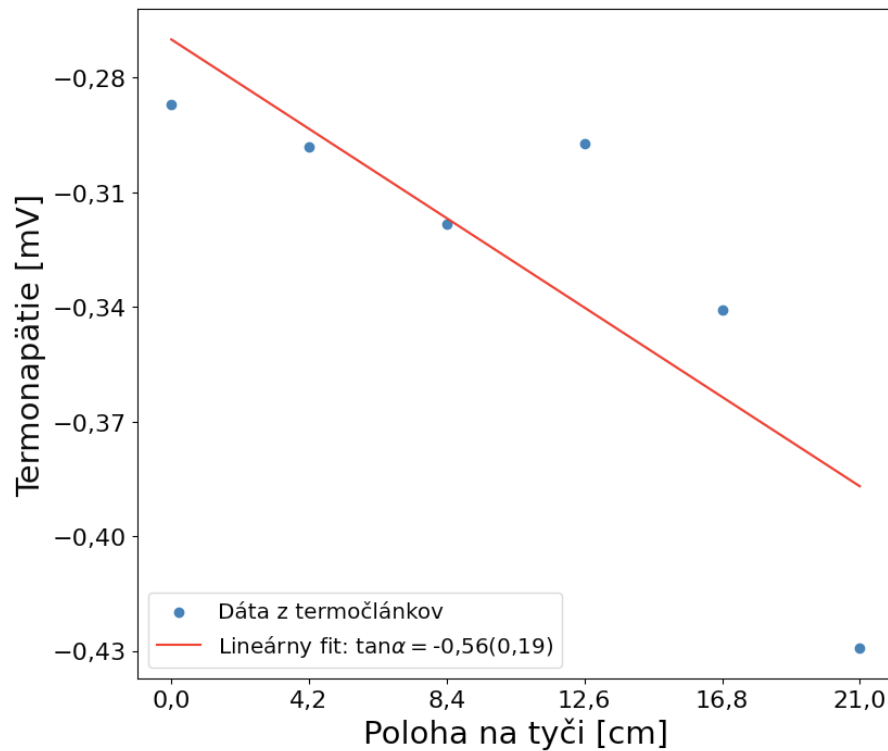
Vykreslené grafy boli tieto (Obr. 4, 5, 6).



Obr. 4: Ustálený stav na začiatku merania



Obr. 5: Ustálený stav po zahriatí



Obr. 6: Ustálený stav po schladení

Z grafov vidíme, že ustálený stav na začiatku (Obr. 4) je identický so stavom na konci (Obr. 6), smernice ich fitov sa po zaokrúhlení rovnajú aj s ich neistotami ($\tan\alpha = -0,56(0,19)$). Tento ustálený stav, ako je vidno z grafov, nespĺňa dostatočne predpoklad lineárnosti pre interpoláciu. Stav po zahriatí (Obr. 5) však túto podmienku spĺňa veľmi dobre, preto je vhodný pre fit touto simuláciou, na to sme použili kód nižšie.

```

1 data = pd.read_csv('chladnutie.txt', sep = '\t', decimal=',', header=None)
2 poc_hodnoty = np.array([data[1][0], data[3][0], data[4][0],
3 data[5][0], data[6][0], data[8][0]], dtype=float)
4
5 slope, intercept, stderr = fit_plot(poz_clanky, poc_hodnoty)
6
7 tau = np.array(data[0], dtype=float)
8 t = np.array([data[1], data[3], data[4], data[5], data[6], data[8]], dtype=float)
9 y_data = t.flatten()
10 delta_tau = (tau[-1] - tau[0]) / len(tau)
11 # simulacia s vyuzitim interpolacie
12 popt, pcov = curve_fit(simulacia2, np.zeros_like(y_data), y_data,
13                        p0=[1.16e-4], # odhad teplotnej vodivosti - tabulkova hodnota
14                        sigma=sigma(y_data), # standardna neistota nameranych dat
15                        absolute_sigma=True)
16
17 perr = np.sqrt(np.diag(pcov)) # standardna neistota
18
19 podmienka(popt[0], delta_x2, delta_tau)
20 print(popt[0], ' ', perr[0]) # hodnota teplotnej vodivosti
21 print(popt[0]*c*rho, ' ', perr[0]*c*rho) # hodnota tepelnej vodivosti

```

Hodnotu súčiniteľa tepelnej vodivosti sme pri simulácii s interpoláciou pre chladnutie dostali $\lambda = 430(30) \text{ W m}^{-1} \text{ K}^{-1}$. Táto hodnota bola získaná pri nastavenom delení $dx = 2,1 \text{ cm}$ pri jemnejšom

delení už nebola zachovaná podmienka stability. Toto delenie neposkytuje o moc väčší rozdiel oproti deleniu z predchádzajúcej simulácie $dx = 3,5 \text{ cm}$, väčšie zjemnenie by bolo možné keby zabezpečíme zaznamenávanie teploty v kratších časových intervaloch alebo meriame materiál, ktorý je horší tepelný vodič. S jemnejším delením by sme mali dosiahnuť vyššiu presnosť merania.

6. Záver

Najbližší výsledok k tabuľkovej hodnote pre súčiniteľ tepelnej vodivosti $\lambda = 395 \text{ Wm}^{-1}\text{K}^{-1}$ [2] sme získali pri prvej simulácii a dátach z chladnutia $\lambda = 410(30) \text{ Wm}^{-1}\text{K}^{-1}$. Výsledok simulácie z dát ohrievania bol pri prvej simulácii značne rozdielny od tabuľkovej hodnoty $\lambda = 193(6) \text{ Wm}^{-1}\text{K}^{-1}$, pre celé meranie bol výsledok prvej simulácie $\lambda = 290(9) \text{ Wm}^{-1}\text{K}^{-1}$.

Pri druhej simulácii s interpoláciou počiatočnej podmienky, bolo možné použiť iba dáta z chladnutia, dostali sme výsledok $\lambda = 430(30) \text{ Wm}^{-1}\text{K}^{-1}$, tento výsledok však presnosť merania oproti prvej simulácii napriek našim očakávaniam nezvýšil, ale ešte viac sa vzdialil tabuľkovej hodnote.

Vplyv efektov výmeny tepla s okolím je jednoznačne viditeľný na linearite rozloženia teploty v tyči pri ustálených stavoch. Stavby pri zachladenej tyči (Obr. 4, 6) vykazujú slabú linearitu, pri dostatočne izolovanom experimente by mali vykazovať linearitu blízku tej ustáleného stavu po zahriatí (Obr. 5).

Aj napriek širokému intervalu našich výsledkov zapríčinenému nedostatočnou tepelnou izoláciou experimentu od okolia sa nám podarilo dokázať, že meď je veľmi dobrý tepelný vodič.

Prílohy

Laboratórny denník

Jupyter notebook *simulacia1.ipynb* s prvou simuláciou

Jupyter notebook *simulacia2.ipynb* s druhou simuláciou

súbor *data.txt* s dátami z celého merania

súbor *ohrev.txt* s dátami z ohrievania

súbor *chladnutie.txt* s dátami z chladnutia

Literatúra

- [1] Fyzikálny kabinet FyzKAB. *Hustota látok* [online]. 2017 [cit. 2025-05-16]. Dostupné z: <http://kabinet.fyzika.net/studium/tabulky/hustota-latek.php>
- [2] E-konstruktor. *Měrná tepelná kapacita, součinitel tepelné vodivosti látek* [online]. 2014 [cit. 2025-05-16]. Dostupné z: <https://e-konstrukter.cz/prakticka-informace/merna-tepelna-kapacita-soucinitel-tepelne-vodivosti-latek>